Quantitative exercise: a pilot benchmark of ClickHouse and DuckDB under an 8 GiB, four-core budget

Authors

Weizhuo Sun <weizhuo@kth.se>

Weiyin Fang <weiyinf@kth.se>

1 Introduction

Our project compares a server-based column store (ClickHouse) with an embedded engine (DuckDB) for gigabyte-scale analytics under one fixed budget of 8 GiB memory and four cores. It asks about query latency (RQ1), memory and disk footprint (RQ2), and load time and the number of query runs needed to pay it back (RQ3). For this exercise we built the measurement set-up and ran a small pilot of the planned experiment; all data below come from it. Its main use is to decide which data the full study must collect, and how much.

2 Dataset (a)

Workload. We generated TPC-H data [1] at scale factors 1 and 10 (SF1, SF10) with the `dbgen` function of DuckDB's `tpch` extension, and exported each of the eight tables to Parquet. SF1 has 8,661,245 rows (0.32 GB of Parquet) and SF10 has 86,586,082 rows (3.42 GB). The row counts of all tables match the TPC-H specification. Both engines load the same files. We use the eight queries of our research plan, grouped by their main access pattern: scan (Q1, Q6), join (Q5, Q9, Q21) and aggregation (Q3, Q13, Q18). The query texts are the TPC-H queries with the default substitution parameters, taken from DuckDB's `tpch` extension. ClickHouse runs all of them unchanged except one: Q13 needs `SETTINGS join_use_nulls = 1`. Without it ClickHouse fills unmatched rows of the outer join with 0 instead of NULL, and the result is wrong.

Systems and set-up. Table 1 lists the configuration. Each engine runs in its own Docker container with `--memory=8g --memory-swap=8g --cpus=4`, pinned to the same four virtual CPUs (`--cpuset-cpus=0-3`); during measurements only one engine container runs at a time. DuckDB is embedded, so its container runs a small Python process that holds the connection, as an application would; a driver container on the other four virtual CPUs runs the experiment. The host is a laptop (Apple M1 Pro, 8 cores, 16 GiB) running Docker Desktop (Docker Engine 29.0.1), whose Linux virtual machine has 10240 MiB of memory and kernel 6.12.54-linuxkit.

Table 1: Fixed configuration. Memory settings were chosen so that each engine stays below the container limit.

	ClickHouse	DuckDB
Version	26.8.36.3 (LTS), official image	2.0.0, Python API in `python:3.12-slim`
Container	8 GiB memory, no swap, 4 CPUs	same
Engine memory limit	`max_server_memory_usage` = 7 GiB	`memory_limit` = 7 GiB
Threads	`max_threads` = 4	`threads` = 4
Spilling	GROUP BY, ORDER BY above 3 GiB; joins in memory	to `/tmp/duck_spill`
Storage, caches	MergeTree on TPC-H primary key; query (condition) cache off	one `.duckdb` file; no result cache

Procedure. For each engine and SF we loaded the data three times. Each load starts from a restarted engine and an emptied page cache of the Linux virtual machine (the cache of the macOS host below it

could not be emptied), imports all tables and is timed. For ClickHouse the background merges are then forced with `OPTIMIZE ... FINAL` and timed separately; for DuckDB the `CHECKPOINT` is timed. For each engine, SF and query we then restarted the engine, emptied the page cache and ran the query once (*cold*), followed by 20 runs in a row (*warm*). This gives 672 query runs and 12 loads. Latency is measured at the client for ClickHouse (HTTP request until the full result is received) and in-process around execute and fetch for DuckDB; Table 3 reports these values. Query order was shuffled with a fixed seed. Before any timing, a correctness check compared every result at SF1 with the TPC-H reference answers, and compared the two engines with each other at both SFs. All checks passed, and every timed run returned the verified result.

3 Variables and expected properties (b)

Table 2 lists the variables. We wrote down the expected properties below before the analysis, and we did not change them afterwards.

Table 2: Independent and dependent variables.

Variable	Type	Values or unit
Engine	independent, categorical	ClickHouse, DuckDB
Scale factor (SF)	independent, ordinal	1, 10
Query class	independent, categorical	scan, join, aggregation (query as blocking factor)
Phase	independent, categorical	cold (first run after restart), warm
Latency	dependent, continuous	ms, wall-clock per query run
Peak memory	dependent, continuous	bytes, maximum of the container's memory during a run
Load time	dependent, continuous	s, import (plus merge or checkpoint) of all tables
Disk footprint	dependent, continuous	bytes on disk after loading

E1. Latency is positive and right-skewed with a long tail.

E2. Cold runs are clearly slower than warm runs.

E3. Warm runs vary little: coefficient of variation (CV) below 10 %.

E4. Latency grows roughly linearly with SF, so the slope in log–log coordinates is about 1.

E5. ClickHouse is faster on join and aggregation queries.

E6. DuckDB loads faster and uses less memory when idle.

E7. Peak memory grows with SF but stays below the 7 GiB engine limit; above it, the engines spill to disk instead of failing.

E8. ClickHouse's compressed disk footprint is smaller than DuckDB's file, or similar.

4 Data analysis (c)

Preprocessing. The driver writes one CSV row per run, and we analyse the files in Python (pandas, SciPy, statsmodels, matplotlib). A script checks that all 672 runs and 12 loads are present, every result is correct, every latency is positive and no peak exceeds 8 GiB. No run failed and no row was removed.

Measurement of memory. ClickHouse is a server while DuckDB shares a process with its host, so there is no common process boundary; we measure both at the container. The driver reads the container's `memory.current` every 2 ms during each run (actual median interval 3.5 ms, maximum 4.8 ms) and takes the maximum. As a check, it also resets and reads the kernel's exact high-water mark (`memory.peak`). The sampled maximum is within 1 % of it in 99.7 % of the runs and never more than 2.0 % below it. This container-level number includes the page cache of files the engine has read

or written, so we also record the anonymous (heap) memory, and each engine's own number: the `memory_usage` in ClickHouse's query log, and the resident set size of the DuckDB process.

Exploratory analysis. We plotted every run (Figure 1), tested normality with the Shapiro–Wilk test, computed the CV and checked each expectation against the data.

Statistical methods. Because latency is skewed (Section 5), we use medians and nonparametric methods [2].

1. For each engine, SF and query: median of the 20 warm runs with a 95 % percentile-bootstrap confidence interval (CI; 10 000 resamples, fixed seed).
2. Engine comparison per SF and query: ratio of medians (ClickHouse/DuckDB) with a bootstrap CI, two-sided Mann–Whitney U test with Holm correction over the eight queries of each SF, and Cliff's δ as effect size [4].
3. Scaling model: for each engine and query we fit $\log t = a + b \log \mathrm{SF}$. With only two SF values the line passes exactly through both medians, so it has no residuals and cannot be tested; the CI of b only reflects run-to-run noise.
4. Break-even (RQ3): the total cost of loading once and running the query set n times is $T(n) = L + n\,q$, where L is the median load time (import plus merge or checkpoint) and q the sum of the eight median warm latencies. With A = ClickHouse and B = DuckDB, the lines cross at $n^* = (L_A - L_B)/(q_B - q_A)$; if $n^* \le 0$, one engine is cheaper for every n.
5. How much data is needed: we recomputed the 95 % CI of the median from only the first n warm runs (n = 3, 5, 6, 10, 15, 20) and looked for the smallest n with a relative half-width below 5 %. We use the exact distribution-free CI based on order statistics [3] instead of the bootstrap CI, because the bootstrap CI is far too narrow for tiny samples: at $n = 3$ it is the range of the three values and contains the true median only 75 % of the time (confirmed in a simulation test). The exact 95 % CI needs $n \ge 6$, so n = 3 and 5 are shown only with the bootstrap.
6. Searching for a model (PCA): we built a matrix with one row per query and eight columns (log median latency and log median peak memory for each engine and SF), standardised the columns and ran a principal component analysis, to see whether the queries group by our three classes.

5 Results (d)

Latency (RQ1). Figure 1 shows all warm runs and the cold run. Contrary to E5, DuckDB was faster in 13 of the 16 comparisons, by a factor of 1.26 to 4.01 (Table 3). ClickHouse was faster only on Q13 at both SFs (ratio 0.89 and 0.92) and on Q18 at SF10, where the difference (ratio 0.98, CI 0.93–1.02) is not significant (Holm-adjusted $p = 0.29$). The other 15 differences are significant (adjusted $p < 0.001$) with a large effect size. Because the 20 runs are consecutive repeats on one machine, this only shows that the differences are larger than the run-to-run noise of this set-up. We did not test why ClickHouse was slower; possible reasons are our default settings, the limit of four threads, and `Decimal` arithmetic on the price columns. The gap depends more on the query than on its class: it is largest for Q1 and Q21 (ratio at least 2.97) and smallest for Q9, Q13 and Q18. With one cold run per query the cold differences cannot be tested, and the macOS host cache may have kept some data warm. The cold run ranked the engines differently from the warm runs for Q13 at SF1 and for Q6, Q3, Q18 at SF10; at SF10 ClickHouse's cold run was faster for Q6, Q3, Q13 (ratio 0.87–0.93). ClickHouse's HTTP interface adds little: its client latency exceeds the server's own query time by a median of 0.8 ms.

Distribution and noise (E1–E3). All 32 warm distributions are right-skewed, and the Shapiro–Wilk test rejects normality for 26 of them, so medians are the right summary (E1 holds). The cold run took 1.3 to 5.2 times the warm median (median 2.3; E2 holds). E3 holds only partly: the CV was below 10 % in 19 of 32 combinations and was up to 23 %. In most combinations the variation comes from a few slow outliers (Figure 1), not from a wide spread.

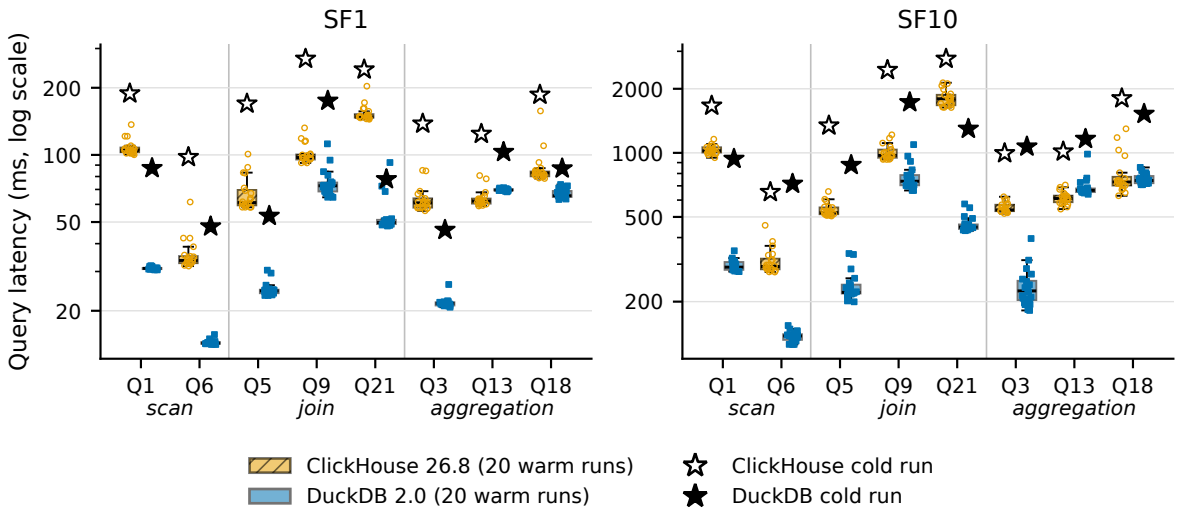


Figure 1: Warm latency per query (box: quartiles and median; points: the 20 runs) and the cold run (star) for each engine. Log scale; note the different axes for SF1 and SF10.

Scaling (E4). Figure 2 shows that the lines of all queries run nearly parallel to the slope-1 reference. The fitted slopes lie between 0.93 and 1.08 (median 0.98), so latency grows about linearly with data size, consistent with E4. The CI of b excludes 1 for 9 slopes below 1 and 2 above 1 (largest: ClickHouse Q21, 1.08), but with two SF points we cannot tell whether this is a real deviation from linearity or a fixed per-query cost.

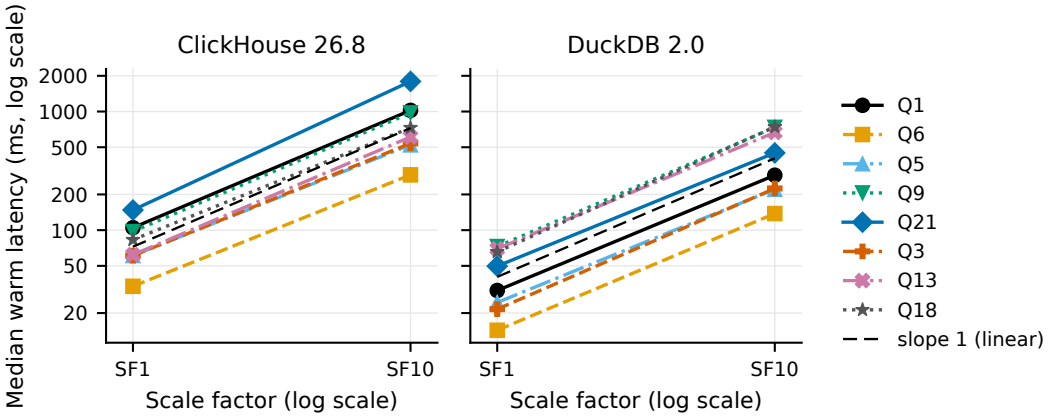


Figure 2: Median warm latency against scale factor (both axes logarithmic). The thick grey line has slope 1 (latency proportional to SF).

Memory (RQ2, E6–E7). Figure 3 shows the peak container memory. DuckDB's container needed much less memory: ClickHouse's median container peak was 4.6–7.0 times DuckDB's at SF1 and 1.6–2.8 times at SF10. A large part of this is the idle footprint: right after a restart the ClickHouse container already used 1.28 GiB, against 0.08 GiB for DuckDB (SF1; E6 holds for memory). Peak memory grew with SF for every engine and query. The highest peak of any query run was 4.89 GiB (ClickHouse, Q18, SF10), well below the 7 GiB limit, and no run failed. The engines' spill counters (ClickHouse's query log, the size of DuckDB's temporary directory) show that 0 runs spilled to disk. E7 therefore holds in part: its second half (spilling instead of failing) was not tested. The two engines' own numbers are smaller than the container peaks, because the container also counts page cache and, for ClickHouse, the idle server: the largest median was 1.69 GiB in ClickHouse's query log and 1.44 GiB of DuckDB process memory (SF10).

Table 3: Warm latency per query: median of 20 runs [95 % bootstrap CI] in ms, ratio of medians ClickHouse/DuckDB [95 % CI], Holm-adjusted p of the Mann–Whitney test and Cliff's δ (negative: ClickHouse faster).

SF	Query	Class	ClickHouse (ms)	DuckDB (ms)	Ratio	p_{Holm}	δ
1	Q1	scan	105 [103, 107]	31.0 [30.8, 31.1]	3.39 [3.32, 3.47]	<0.001	1.00
1	Q6	scan	33.6 [32.8, 35.0]	14.3 [14.2, 14.4]	2.35 [2.29, 2.45]	<0.001	1.00
1	Q5	join	61.1 [59.6, 68.2]	24.6 [24.1, 24.8]	2.49 [2.42, 2.78]	<0.001	1.00
1	Q9	join	97.3 [96.1, 100]	72.7 [68.7, 74.4]	1.34 [1.31, 1.43]	<0.001	0.90
1	Q21	join	148 [147, 151]	49.7 [49.0, 50.9]	2.97 [2.90, 3.05]	<0.001	1.00
1	Q3	aggregation	60.9 [58.1, 62.9]	21.6 [21.3, 21.8]	2.82 [2.68, 2.93]	<0.001	1.00
1	Q13	aggregation	62.1 [60.7, 63.4]	69.6 [69.4, 69.9]	0.89 [0.87, 0.91]	<0.001	−0.80
1	Q18	aggregation	83.0 [80.7, 83.7]	65.6 [64.9, 68.4]	1.26 [1.20, 1.28]	<0.001	1.00
10	Q1	scan	1022 [1005, 1051]	290 [283, 303]	3.52 [3.35, 3.65]	<0.001	1.00
10	Q6	scan	292 [285, 313]	138 [134, 139]	2.12 [2.06, 2.29]	<0.001	1.00
10	Q5	join	523 [516, 544]	221 [219, 236]	2.37 [2.21, 2.46]	<0.001	1.00
10	Q9	join	973 [957, 1020]	736 [706, 778]	1.32 [1.25, 1.42]	<0.001	0.88
10	Q21	join	1795 [1690, 1853]	448 [441, 459]	4.01 [3.72, 4.13]	<0.001	1.00
10	Q3	aggregation	539 [533, 565]	225 [206, 244]	2.40 [2.20, 2.68]	<0.001	1.00
10	Q13	aggregation	610 [587, 627]	667 [655, 677]	0.92 [0.88, 0.95]	<0.001	−0.78
10	Q18	aggregation	730 [706, 751]	741 [722, 768]	0.98 [0.93, 1.02]	0.29	−0.20

Load time and disk (RQ2–RQ3, E6, E8). Table 4 lists the loads. DuckDB loaded 3.0 times faster at SF1 and 3.3 times faster at SF10 (E6 holds). For ClickHouse, the forced merge takes almost as long as the import itself, while DuckDB's final checkpoint at SF10 takes only milliseconds because DuckDB already writes checkpoints during the import. E8 does not hold: ClickHouse's tables took 1.57 (SF1) and 1.69 (SF10) times the space of the DuckDB file, and more than the Parquet input, while the DuckDB file was smaller than the Parquet input. A likely reason is that ClickHouse by default compresses with LZ4, which favours speed over size, but we did not test this. During the SF10 loads the container memory of both engines reached the 8 GiB limit. This is page cache of the files being written, which the kernel reclaims; the anonymous memory stayed at or below 3.07 GiB and no load failed.

Table 4: Loads (3 per engine and SF). Each column is the median of its own three values, so Total need not equal Import + Merge/ckpt: import time (min–max), merge (ClickHouse) or checkpoint (DuckDB) time, total; disk footprint and its ratio to the Parquet input (computed from unrounded sizes); peak container / anonymous memory during the load (maximum); container memory right after restart (median over the cold runs). GB = 10^9 bytes, GiB = 2^{30} bytes.

Engine	SF	Import (s)	Merge/ckpt (s)	Total (s)	Disk (GB)	Disk/Parquet	Load peak (GiB)	Idle (GiB)
ClickHouse	1	5.2 (5.0–6.5)	4.6	9.8	0.47	1.44	3.83 / 2.50	1.28
DuckDB	1	3.1 (2.8–3.3)	0.20	3.2	0.30	0.92	1.29 / 0.68	0.08
ClickHouse	10	47.5 (44.4–47.7)	40.5	87.1	4.73	1.38	8.00 / 2.96	1.33
DuckDB	10	26.7 (25.8–27.5)	0.003	26.7	2.80	0.82	8.00 / 3.07	0.09

Break-even (RQ3). DuckDB both loads and queries faster, so the cost lines never cross for $n > 0$ (Figure 4). The formula gives $n^* = -21.6$ at SF1 and -20.0 at SF10, and none of the bootstrap resamples gave a positive n^*. In this pilot DuckDB is cheaper for any number of runs.

How many runs are needed. Figure 5 shows how the CI of the median narrows as more warm runs are used. With the exact CI, a half-width below 5 % was reached for 28 % of the combinations with 10 runs, 62 % with 15 runs and 84 % with 20 runs. With 20 runs the median half-width was 3.2 % and the largest 9.7 %. For 5 combinations even 20 runs were not enough. The bootstrap line shows why bootstrap CIs from small samples are misleading: at $n = 3$ its median half-width is only 7.1 %, but these intervals miss the true median one time in four.

Searching for structure (PCA). The first two components explain 69 % and 17 % of the variance

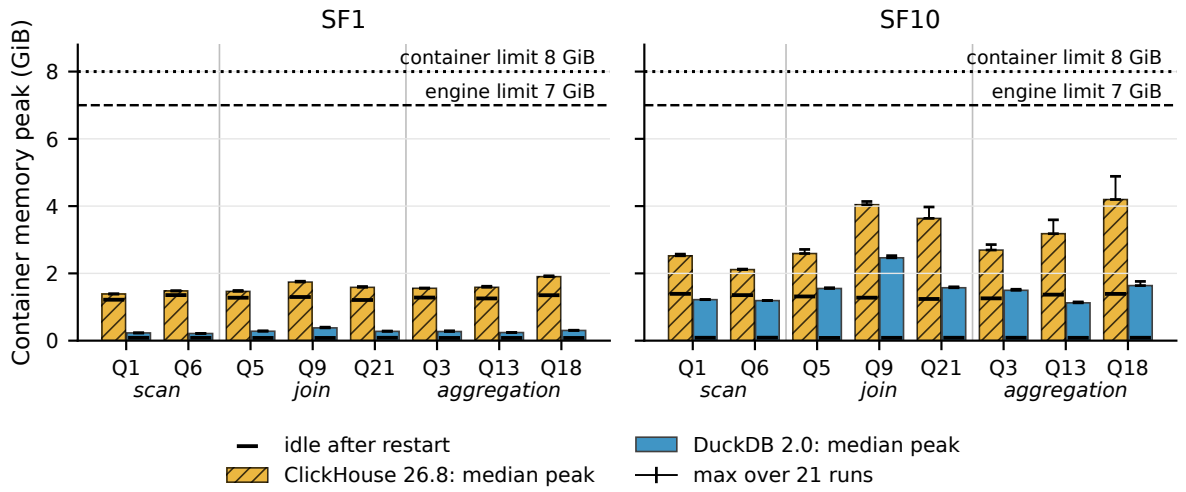


Figure 3: Peak container memory per query (bar: median of the 20 warm runs; whisker: maximum over all 21 runs) and the container's memory right after restart (black dash; for DuckDB only 0.08 GiB at SF1, close to the axis). Dashed and dotted lines: engine and container limits.

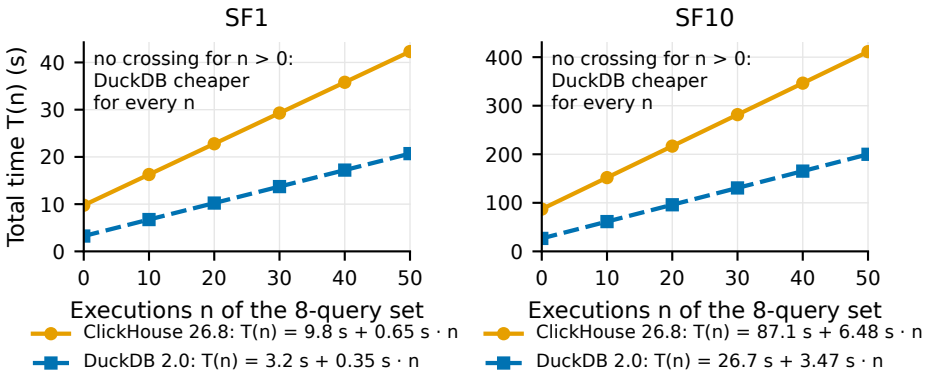


Figure 4: Total cost $T(n)$ of loading once and running the 8-query set n times. Intercept: median load time; slope: sum of median warm latencies.

(Figure 6). The first component loads on all eight columns with the same sign, so it simply orders the queries from cheap (Q6) to expensive (Q9, Q18). The second separates queries that are slow in ClickHouse (Q1, Q21) from queries whose memory is high relative to their latency (Q6, Q9, Q18). The queries do not group by our three classes: the two scan queries are far apart, and the join queries Q5 and Q9 lie on opposite sides of the first component. With only eight queries this is a weak test. A better model would describe each query by its cost drivers (rows scanned, join sizes, number of groups) rather than by one class.

6 Data needed for the full experiment

Table 5 compares what the pilot collected with what we now think the full study needs, and why.

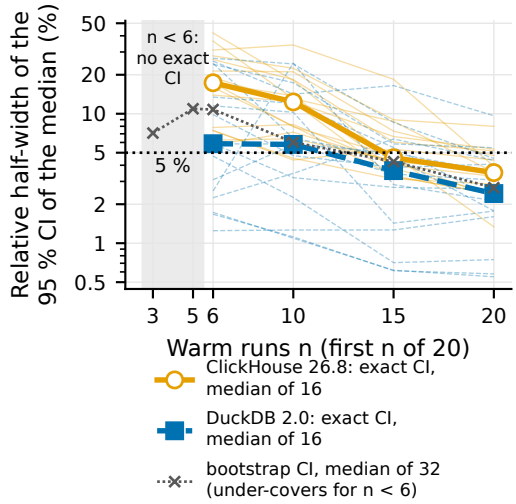


Figure 5: Relative half-width of the 95 % CI of the median when only the first n warm runs are used (thin lines: each engine, SF and query; thick lines: median per engine).

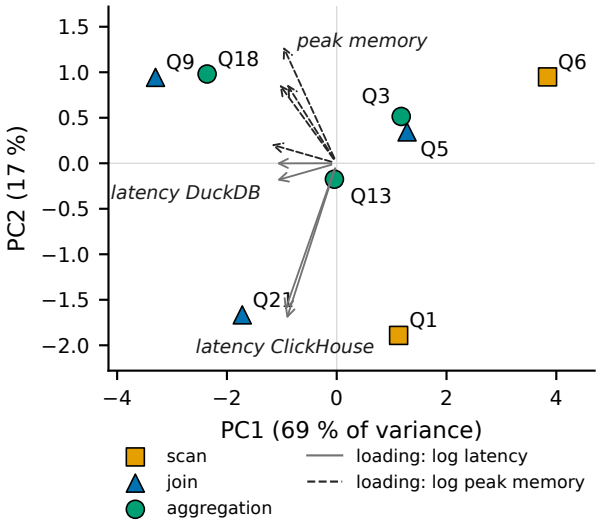


Figure 6: PCA biplot of the eight queries; arrows show the loadings of the eight columns.

Table 5: Data collection in the pilot and in the full study.

Item	Pilot	Full study	Reason (from this pilot)
Warm runs per combination	20	at least 20; continue until the exact-CI half-width is below 5 %, at most 40	20 runs gave a CI half-width below 5 % for 84 % of the combinations; 10 runs only for 28 %
Cold runs per combination	1	8 (at least 6)	one run gives no CI; an exact 95 % CI of a median needs at least 6 runs
Loads per engine and SF	3	at least 5	with 5 against 5 loads the smallest two-sided Mann–Whitney p is 0.008, below the Holm threshold for two SFs
Scale factors	1, 10	1, 3, 10	two points always fit the scaling model; a third point gives a residual to test it
Memory pressure	none	also SF30, or a 4 GiB budget	the highest query peak was 4.89 GiB and no run spilled
Concurrency	one query at a time	also two and four clients	in our qualitative exercise, both simulated practitioners named the number of users querying the data as the deciding factor
Configuration	defaults	plus one ClickHouse variant	`Float64` instead of `Decimal` or stronger compression may change RQ1 and RQ2
Machine	laptop, Linux VM	dedicated Linux machine	no hidden macOS page cache; as in the research plan
Total query runs	672	at least 1344	2 engines × 3 SFs × 8 queries × (8 cold + 20 warm); the pilot's loads took 7 and its queries 6 minutes

7 Discussion (e)

Did the results match our expectations? Partly. Skewed latency, slow cold runs and linear scaling all appeared, and DuckDB loaded faster and used less memory, as expected. The main surprise is RQ1: we expected ClickHouse to be faster on join and aggregation queries, but DuckDB was faster in 13 of 16 comparisons. For RQ2, ClickHouse's tables were also larger on disk than we expected. Warm runs were also noisier than we assumed.

Which data were most useful? The warm runs answered RQ1, the idle and peak container memory

explained most of RQ2 (much of ClickHouse's extra memory is its idle footprint), and the load times decided RQ3.

Would more data help? Yes, but of specific kinds (Section 6): more cold runs and loads to make those comparisons testable, a third scale factor to test the scaling model, conditions that cause spilling, and concurrent queries. More warm runs of the same kind would add little, except for the few noisy combinations.

How general and accurate are the results? The numbers are accurate for this set-up (every result was verified, the memory sampling agrees with the kernel's exact high-water mark, and most CIs are narrow), but they are not general. We used one laptop with a Linux virtual machine under macOS (whose own page cache we could not empty), one run order, two SFs and default settings with four threads and `Decimal` columns, which may keep ClickHouse far from its best. The engines' own memory numbers also measure different things. The pilot therefore shows which data the full study needs; it does not yet show which engine is better in general.

8 Reproducibility

All scripts, configuration, query texts, raw CSV files and this report are in our repository; `make all` rebuilds everything from data generation to this PDF, and a script checks that every number in the report comes from `results/stats/*.csv`.

Use of AI tools. An AI assistant (Claude, Anthropic) wrote most of the measurement and analysis scripts and drafted this report under our direction. We reviewed the code, the data and the text, and we take responsibility for them.

References

[1] Transaction Processing Performance Council, *TPC Benchmark H Standard Specification*, Revision 3.0.1. [Online]. Available: https://www.tpc.org/tpch/

[2] T. Hoefler and R. Belli, "Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results," in *Proc. SC '15*, ACM, 2015, pp. 73:1–73:12. doi: 10.1145/2807591.2807644

[3] J.-Y. Le Boudec, *Performance Evaluation of Computer and Communication Systems*. Lausanne: EPFL Press, 2010.

[4] J. Romano, J. D. Kromrey, J. Coraggio, and J. Skowronek, "Appropriate statistics for ordinal level data: Should we really be using t-test and Cohen's d for evaluating group differences on the NSSE and other surveys?" in *Annual Meeting of the Florida Association of Institutional Research*, 2006.